

Regular expressions

Linux

Basic Syntax

- `/.../`: Start and end regex delimiters
- `|`: Alternation
- `()`: Grouping

Character Classes

- `\s`: Whitespace
- `\S`: Not whitespace
- `\w`: Word
- `\W`: Not word
- `\d`: Digit
- `\D`: Not digit
- `\x`: Hexadecimal digit
- `\O`: Octal digit

Groups and Ranges

- `.`: Any character except newline (`\n`)
- `(alb)`: a or b
- `(...)`: Group
- `(?:...)`: Passive (non-capturing) group
- `[abc]`: a, b or c
- `[^abc]`: Not a, b or c
- `[a-z]`: Letters from a to z
- `[A-Z]`: Uppercase letters from A to Z
- `[0-9]`: Digits from 0 to 9

Quantifiers

- $*$: 0 or more
- $+$: 1 or more
- $?$: 0 or 1
- $\{3\}$: Exactly 3
- $\{3, \}$: 3 or more
- $\{3, 5\}$: 3, 4 or 5

Expression

`/\b[eSR]\w+/g`

Text

Tests

NEW

RegExp was created by gskinner.com.

Edit the Expression & Text to see matches. Roll over matches or the expression flavors of RegExp are supported. Validate your expression with Tests mode.

The side bar includes a Cheatsheet, full Reference, and Help. You can also Save view patterns you create or favorite in My Patterns.

<https://regextir.com/>

Position Matching

- `^`: Start of string or start of line in multi-line mode
- `$`: End of string or end of line in multi-line mode
- `\b`: Word boundary
- `\B`: Not word boundary

Linux

Regular expressions

Basic Syntax

- `/.../`: Start and end
- `|`: Alternation
- `()`: Grouping

Groups and Ranges

- `.`: Any character except newline (`\n`)
- `(alb)`: a or b
- `(...)`: Group
- `(?:...)`: Passive (non-capturing) group
- `[abc]`: a, b or c
- `[^abc]`: Not a, b or c
- `[a-z]`: Letters from a to z
- `[A-Z]`: Uppercase letters from A to Z
- `[0-9]`: Digits from 0 to 9

Quantifiers

- `*`: 0 or more
- `+`: 1 or more
- `?`: 0 or 1
- `{3}`: Exactly 3
- `{3,}`: 3 or more
- `{3,5}`: 3, 4 or 5

Position Matching

- `^`: Start of string or start of line
- `$`: End of string or end of line
- `\b`: Word boundary
- `\B`: Not word boundary

Character Classes

- `\s`: Whitespace
- `\S`: Not whitespace
- `\w`: Word
- `\W`: Not word
- `\d`: Digit
- `\D`: Not digit
- `\x`: Hexadecimal digit
- `\O`: Octal digit



<https://regexr.com/>

Linux

Sed & regex examples

```
$ cat anchors.txt  
sub par  
spar  
apparent effort  
two spare computers  
cart part tart mart
```

```
# words starting with 'par'  
$ sed -n '/\bpar/p' anchors.txt  
sub par  
cart part tart mart
```

```
# words ending with 'par'  
$ sed -n '/par\b/p' anchors.txt  
sub par  
spar
```